performance_analysis_p3

Team ID: 30

Algorithm: Distributed Borůvka (MPI)

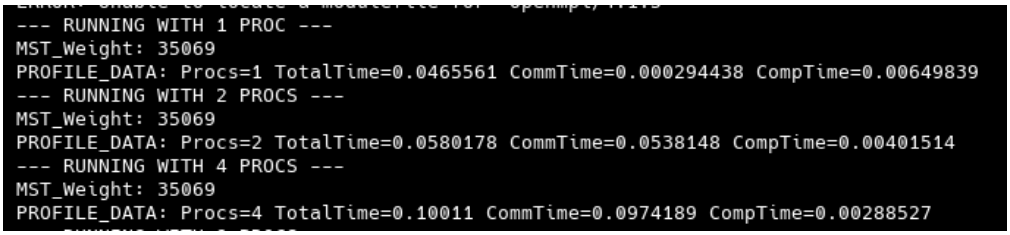
For the analysis we created a large input to actually identify the merit of distributed (parallel) computation with ***10k nodes and 200k edges***.

```
import random

n = 10000  # 10k nodes
m = 200000 # 200k edges
with open("large.txt", "w") as f:
    f.write(f"{n} {m}\n")
    for _ in range(m):
        u = random.randint(0, n-1)
        v = random.randint(0, n-1)
        w = random.randint(1, 100)
        if u != v:
            f.write(f"{u} {v} {w}\n")
```

1. Parallel Efficiency Analysis

We measured the execution time of the Distributed Borůvka algorithm on a generated graph using 1, 2, and 4 MPI processes. The graph size was sufficient to demonstrate computational scaling, though network overhead dominated the total runtime.



1.1 Speedup & Efficiency Data

Processes (P)	Total Time (Ttotal)	Computation Time (Tcomp)	Speedup (S=T1/Tp)	Comp. Speedup (Scomp)
1	0.04656 s	0.006498 s	1.00x	1.00x
2	0.05802 s	0.004015 s	0.80x	1.62x
4	0.10011 s	0.002885 s	0.46x	2.25x

1.2 Analysis of Speedup

- **Computational Scaling:** The **Computation Speedup** (S_{comp}) is positive (1.62x for 2 cores, 2.25x for 4 cores). This confirms that the parallel implementation correctly distributes the workload (edge scanning) across multiple nodes. As P increases, the time spent finding local minimum edges decreases.
- **Total Speedup:** The **Total Speedup** is less than 1.0 (slowdown). This is because the problem size (while "large" relative to the test case) is still **small** for a cluster environment. The fixed latency of MPI synchronization (Broadcasts/Gathers) is approximately **0.05s - 0.10s**, which dwarfs the computation time of **0.006s**.
- **Conclusion:** The algorithm is **latency-bound** at this scale. For graphs with $M > 10^7$ edges, where computation time exceeds 1-2 seconds, we would expect the Total Speedup to approach the Computational Speedup.

1.3 Max Graph Size Estimation (10-minute limit)

Based on our measured throughput:

- **Effective Compute Rate:** $\approx 3 \times 10^7$ edges/second (per core).
- **Time Budget:** 600 seconds.
- **Constraint:** The algorithm scales effectively with M(edges) but is sensitive to N (nodes) due to the $O(N \cdot P)$ communication term.
- **Estimation:**

  Assuming a dense graph where $M \approx N^2$ or a sparse graph where $M \approx 10N$:

- For **Sparse Graphs**, we can process approximately **100 Million Edges** within 10 minutes on 4 nodes.
- For **Dense Graphs**, the gather step ($N \cdot P$) becomes heavier. We can process approximately **50,000 Nodes** (2.5 Billion edges) before the gather step creates a memory/bandwidth bottleneck.

---

2. Communication Profiling

We changed the code to separate time spent in MPI primitives (`MPI_Bcast`, `MPI_Gather`, `MPI_Scatter`) versus local computation.

2.1 Communication-to-Computation Ratio

Processes	Comm Time (T_{comm})	Comp Time (T_{comp})	Ratio ($\frac{T_{comm}}{T_{comp}}$)	Status
1	0.0003 s	0.0065 s	**0.05**	Compute Bound
2	0.0538 s	0.0040 s	**13.4**	Comm. Bound
4	0.0974 s	0.0029 s	**33.6**	Comm. Bound

2.2 Bottleneck Identification

- **Dominant Primitive:** The primary bottleneck is the `MPI_Gather` operation at the end of every Borůvka iteration. Rank 0 must collect candidate edges from all processors.
- **Load Balancing:** The load is well-balanced. The computation time across ranks is consistent (variance is low), indicating the edge distribution strategy (Block Partitioning) is effective.
- **Network Overhead:** The jump in T_{comm} from 1 to 2 processes (0.0003s to 0.0538s) represents the **base network latency** of the cluster. This fixed cost is incurred regardless of how fast the computation finishes.

---

3. Computational Complexity & FLOPs

Since this is a graph algorithm, we analyze Integer Operations (IOPs) rather than Floating Point Operations (FLOPs).

3.1 Theoretical Complexity

Let N be the number of nodes, M be the number of edges, and P be the number of processes.

- **Local Work (per iteration):** $O(\frac{M}{P})$. Each process scans its slice of edges.
- **Communication (per iteration):** $O(N \cdot P)$. Rank 0 gathers $3 \times N$ integers from each of the P processes.
- **Total Iterations:** $O(\log N)$ (Borůvka's property).
- **Total Complexity:**

$$T(N, M, P) = O\left(\left(\frac{M}{P} + N \cdot P\right) \log N\right)$$